

第 8 章：Actor-Critic：从 TD Critic 到 Advantage、n-step 与 GAE

8.1 本章符号说明

符号/缩写	English	中文	含义
AC	Actor-Critic	行动者-评价者框架	Actor 学策略，Critic 学价值并给 Actor 提供学习信号。
A2C	Advantage Actor-Critic	优势 Actor-Critic	多环境同步采样、统一更新的 on-policy Actor-Critic 算法。
A3C	Asynchronous Advantage Actor-Critic	异步优势 Actor-Critic	多个 worker（工作进程或线程）异步采样并更新共享参数的算法。
TD	Temporal Difference	时序差分	使用 reward 加下一状态价值进行 bootstrap 的估计方法。
GAE	Generalized Advantage Estimation	广义优势估计	对多步 TD residual 加权，构造 advantage estimate。
MC	Monte Carlo	蒙特卡洛	使用完整实际 return、不 bootstrap 的估计方法。
$S_t / A_t / R_{t+1}$	State / Action / Reward Random Variable	状态 / 动作 / 奖励随机变量	时刻 t 的状态、动作和执行动作后收到的奖励。
$\pi_\theta(a s)$	Actor Policy	Actor 策略	参数为 θ 的动作分布。
θ	Actor Parameters	Actor 参数	策略网络的可学习参数。
$V_w(s)$	Critic State-value Estimate	Critic 状态价值估计	参数为 w 的价值网络输出。
w	Critic Parameters	Critic 参数	价值网络的可学习参数。
α_w	Critic Learning Rate	Critic 学习率	控制每次 Critic 参数更新的步长。
$V_\pi(s) / Q_\pi(s,a)$	True Value Functions	理论价值函数	策略 π 下的真实条件期望，用于推导。
$A_\pi(s,a)$	Advantage Function	理论优势函数	$Q_\pi(s,a) - V_\pi(s)$ 。
$\hat{A}_t$	Advantage Estimate	优势估计量	实际更新 Actor 时使用的带噪声估计；帽子表示它不是真实 A_π 。
γ	Discount Factor	折扣因子	控制未来 reward/value 的权重。
d_t	Terminal Indicator	终止指示量	真正终止时为 1，此后没有环境定义的未来价值。
$1-d_t$	Bootstrap Mask	自举掩码	终止时为 0、继续时为 1，控制是否加入下一状态价值。
$Y_t^{(1)}$	One-step Value Target	一步价值目标	$R_{t+1} + \gamma(1-d_t)V_w(S_{t+1})$ 。
$Y_t^{(n)}$	n-step Value Target	n 步价值目标	连续 n 步实际 reward 加第 n 步 bootstrap value。
δ_t	TD Residual / TD Error	TD 残差 / 时序差分误差	一步 bootstrap target 与当前 $V_w(s_t)$ 的差。

符号/缩写	English	中文	含义
λ	GAE Lambda	GAE 衰减参数	控制 advantage estimate 看多远。
τ	Trajectory	轨迹	策略与环境交互得到的状态、动作、奖励序列。
B_{roll}	On-policy Rollout Batch	同策略采样批次	当前策略刚采集、用于本轮更新的一批 transition。
$\mathbb{E}_{B_{roll}}[\cdot]$	Empirical Batch Average	批内样本均值	对 B_{roll} 中所有有效 transition 求和再除以样本数，不是只对下标 t 求概率期望。
L_b	Baseline Regression Loss	Baseline 回归损失	用完整 Monte Carlo return 训练第 7 章 baseline。
L_{actor} / L_{critic}	Actor / Critic Loss	Actor / Critic 损失	优化器实际最小化的策略损失和价值损失。
$\hat{V}_t^{target}$	Estimated Value Target	价值目标估计	Critic 在时刻 t 回归的目标值，不是当前网络预测。
$H(\pi)$	Policy Entropy	策略熵	衡量动作分布随机程度。
β_H	Entropy Coefficient	熵系数	控制 entropy bonus 的强度。
c_V	Value-loss Coefficient	价值损失系数	共享总 loss 中 Critic loss 的权重。
$sg(\cdot)$	Stop Gradient	停止梯度	前向使用数值，反向不沿该路径传播梯度。
K_t / U_t	Discrete Type / Continuous Parameter	离散类型 / 连续参数随机变量	混合动作 $A_t = (K_t U_t)$ 的两个组成部分。
θ_d / θ_c	Discrete / Continuous Policy Parameters	离散 / 连续策略参数	混合策略中两个动作分支各自的可学习参数。
$Q_w(s, k, u)$	Action-value Critic Estimate	动作价值 Critic 估计	参数为 w 、同时输入状态和混合动作的价值网络输出。
$N / T_i / M$	Number of Rollouts / Rollout Length / Batch Size	采样片段数 / 第 i 段长度 / 批样本数	$M = \sum_i T_i$ ；固定长度 A2C 中常有 $T_i = T$ 、 $M = NT$ 。
rollout	Policy Rollout	策略采样片段	用当前策略与环境交互得到的一段连续 transition 数据。
transition	One-step Transition	一步转移样本	一次状态、动作、奖励、下一状态和终止标记组成的数据。
on-policy	On-policy Learning	同策略学习	Rollout 由当前正在更新的策略产生。

8.2 从 REINFORCE with Baseline 到 TD Actor-Critic

8.2.1 REINFORCE with Learned Baseline 已经有了什么

第 7 章的 REINFORCE with learned baseline 使用：

$$\hat{A}_t^{MC} = G_t - b_\varphi(s_t)$$

这里 G_t 是从时刻 t 开始的 Return（回报）， $b_\varphi(s_t)$ 是参数为 φ 的 Learned Baseline（学习型基线），上标 MC 表示该优势估计使用完整 Monte Carlo 回报。它已经解决“不能只看回报绝对值，要减去状态平均水平”的问题，但仍要等完整 G_t 才能训练。

从广义上说，只要 $b(s)$ 学习价值并评价 Actor，learned value baseline 就可以叫 Critic。本章所说的典型 TD Actor-Critic，关键新增点不是换一个名字，而是让价值网络通过 bootstrap 提前学习，并持续给 Actor 提供 advantage estimate。

8.2.2 Bootstrap 解决等待完整回报的问题

完整 Monte Carlo (MC, 蒙特卡洛) 回报有两个特点：

- 优点：目标全部来自实际奖励，不含 bootstrap 误差。
- 缺点：必须等待较长未来，且单条轨迹回报方差高。

Actor-Critic 引入 Temporal Difference (TD, 时序差分) Critic：观察一步或若干步实际奖励后，用价值预测补上尚未发生的未来。这样可以更早更新、降低方差，但会引入 Critic 估计偏差。

8.2.3 本章要建立的两条训练线

REINFORCE with learned baseline

- > 训练线一 Critic: Bellman bootstrap -> TD target -> value loss
- > 训练线二 Actor: TD residual -> advantage estimate -> policy loss
- > 两条线扩展: one-step -> n-step -> GAE
- > 完整算法: Actor loss + Critic loss + optional entropy loss
- > 采样组织: A2C 同步 rollout / A3C 异步 rollout

本章不处理“策略一步更新太大”的问题；那是第 9 章 Trust Region Policy Optimization (TRPO, 信赖域策略优化) 和 Proximal Policy Optimization (PPO, 近端策略优化) 的主题。

8.3 Actor-Critic 核心结构：Actor 选动作，Critic 学价值

8.3.1 Actor 与 Critic 的输入、输出和训练目标

最常见的 State-value Actor-Critic (状态价值 Actor-Critic) 包含两个可学习模块：

模块	输入	输出	自己的训练信号	对另一模块的作用
Actor	状态 S_t	动作分布 $\pi_\theta(\cdot/S_t)$	Policy-gradient Loss (策略梯度损失)	决定采样哪些动作与状态
Critic	状态 S_t	价值估计 $V_w(S_t)$	Value Regression Loss (价值回归损失)	产生 advantage estimate

两者不是“Actor 主网络加一个辅助 loss”。Critic 有独立的预测目标和参数更新；Actor 只是使用 Critic 输出构造出的数值评价。

8.3.2 一条 Transition 如何同时服务 Critic 和 Actor

一次环境交互得到 Transition (一步转移样本)：

$(S_t, A_t, R_{t+1}, S_{t+1}, d_t)$

它在 Actor-Critic 中沿两条训练线流动：

Actor 根据 S_t 采样 A_t

- > 环境返回 R_{t+1} 、 S_{t+1} 、 d_t
- > Critic 用 reward + next value 构造 TD target
- > TD residual = target - current value
 - > Critic: 作为价值预测的纠偏误差
 - > Actor: 作为已采样动作的 advantage estimate

所以 Critic 不是先凭空输出一个“动作分数”给 Actor。它先通过自己的价值回归任务学习状态平均水平，再用实际结果相对该平均水平的偏差评价 Actor 的动作。8.4 完整推导 Critic，8.5 再证明这个偏差为什么能训练 Actor。

8.3.3 On-policy Rollout Buffer 是本章的数据来源

基础 Advantage Actor-Critic (A2C, 优势 Actor-Critic) 与 Asynchronous Advantage Actor-Critic (A3C, 异步优势 Actor-Critic) 属于 On-policy Learning (同策略学习)：用于更新的数据来自当前正在学习的策略。

数据容器	数据来源	使用方式	参数更新后
Rollout Buffer (采样缓冲区)	当前策略刚采集的数据	顺序计算 TD、return、GAE 和本轮 loss	通常清空
Replay Buffer (经验回放池)	多个历史策略积累的数据	长期保存并反复随机采样	继续保留

因此本章有临时 rollout buffer，没有 Deep Q-Network (DQN, 深度 Q 网络) 式的 replay buffer。若把旧策略数据直接放进基础 on-policy Actor loss，样本分布不再对应当前 $p_{\theta}(t)$ ，梯度估计会产生分布偏差。使用历史 replay 数据的 Off-policy Actor-Critic (异策略 Actor-Critic) 需要另一套目标或校正方法，留到第 10 章讨论。

8.4 第一条训练线：Critic 用 TD Target 学习 $V_{\pi}(s)$

8.4.1 Critic 的监督目标是当前策略的状态价值

本章先讨论最常见的 State-value Critic (状态价值评价者)。它接收状态 s ，输出：

$$V_w(s) \approx V_{\pi}(s)$$

其中 $V_{\pi}(s)$ 不是“当前局面看起来有多好”的任意评分，而是：从状态 s 出发，之后一直按当前策略 π 行动时，未来折扣回报的条件期望。

$$V_{\pi}(s) = \mathbb{E}_{\pi}[G_t | S_t=s]$$

因此 Critic 学的是当前 **Actor** 对应的价值函数。Actor 的策略变化后， V_{π} 也会变化，所以 Critic 面对的是一个随训练缓慢移动的预测问题。

Critic 的两个用途要分开看：

1. **预测用途**：估计某个状态之后还能获得多少回报。
2. **评价用途**：比较一次实际结果与该状态的平均预期，为 Actor 产生“比预期好还是差”的信号。

8.4.2 从完整 G_t 回归改成 Bellman Bootstrap

第 7 章的 learned baseline 用完整回报训练：

$$L_b(w) = \mathbb{E}_{B_{roll}} [(V_w(S_t) - G_t)^2]$$

这里：

- $L_b(w)$ 是 Baseline Regression Loss (Baseline 回归损失)。
- $\mathbb{E}_{B_{roll}}[\cdot]$ 表示对当前采样批次中的全部有效时间步取平均。
- G_t 是轨迹走完后才能算出的 Monte Carlo Return (蒙特卡洛回报)。

这种 baseline 已在学习 V_{π} ，但它的标签仍是完整 G_t ：等待时间长、样本方差高。Actor-Critic 中的 TD Critic 改用 Bellman 递推关系构造更及时的标签。

Bellman Expectation Equation (贝尔曼期望方程) 为：

$$V_{\pi}(s_t) = \mathbb{E}_{\pi}[R_{t+1} + \gamma V_{\pi}(S_{t+1}) | S_t=s_t]$$

它把一个难以直接观测的长期价值拆成两部分：已经观测到的即时奖励，以及下一状态仍未发生的长期价值。Bootstrap (自举) 就是用 Critic 当前的下一状态预测，近似第二部分。

8.4.3 用一条 Transition 构造一步 TD Target

采样一次 transition (一步交互数据) 后，我们已知：

当前状态 S_t
 实际动作 A_t
 即时奖励 R_{t+1}
 下一状态 S_{t+1}
 是否真正终止 d_t

这里 d_t 是 Terminal Indicator（终止指示量）：

- $d_t=1$ ：执行 A_t 后环境真正终止，之后没有环境定义的未来价值。
- $d_t=0$ ：环境仍可继续。

因此 $(1-d_t)$ 是 Bootstrap Mask（自举掩码）：终止时为 0，继续时为 1。一步价值目标为：

$$Y_t^{(1)} = R_{t+1} + \gamma(1-d_t)V_w(S_{t+1})$$

$Y_t^{(1)}$ 是 One-step Value Target（一步价值目标），上标 (1) 表示向前看一步，不是一次方。它有两类信息来源：

- **真实锚点**：环境给出的 R_{t+1} ，以及终止状态没有后续价值这一边界条件。
- **估计部分**：Critic 自己给出的 $V_w(S_{t+1})$ 。

所以 bootstrap 不是凭空“拿自己的答案教自己”。奖励和终止状态提供锚点，Bellman 递推把这些信息逐步向更早状态传播；代价是下一状态预测有误时，误差也会传播。

8.4.4 Critic Loss 如何更新价值网络参数

构造 Critic loss 前，先定义：

$$\begin{aligned} sg(x) &= x \\ \nabla_x sg(x) &= 0 \end{aligned}$$

sg 是 Stop Gradient（停止梯度）：前向计算保留 x 的数值，反向传播不沿该路径继续求梯度。对单个样本，Critic loss 写成：

$$L_t^{sample}(w) = 1/2 (V_w(S_t) - sg(Y_t^{(1)}))^2$$

L_t^{sample} 是 Single-sample Critic Loss（单样本 Critic 损失），上标 $sample$ 表示它只对应一个样本。设批次 B_{roll} 含 N 段 rollout，第 i 段有 T_i 个有效时间步，总 transition 数为 $M = \sum_{i=1}^N T_i$ 。批量 Critic loss 为：

$$L_{critic}(w) = 1/M \sum_{i=1}^N \sum_{t=0}^{T_i-1} L_{i,t}^{sample}(w)$$

也可以把这个双重求和简写成 $\mathbb{E}_{B_{roll}}[L_t^{sample}]$ 。因为 target 被停止梯度，单样本梯度为：

$$\nabla_w L_t^{sample} = (V_w(S_t) - Y_t^{(1)}) \nabla_w V_w(S_t)$$

定义 TD Residual（时序差分残差，也常叫 TD Error）为：

$$\delta_t = Y_t^{(1)} - V_w(S_t)$$

于是梯度下降更新可改写为：

$$w \leftarrow w + \alpha_w \delta_t \nabla_w V_w(S_t)$$

α_w 是 Critic Learning Rate（Critic 学习率）。这条式子直接说明 Critic 怎么学：

- $\delta_t > 0$ ：target 高于当前预测，更新使 $V_w(S_t)$ 倾向增大。
- $\delta_t < 0$ ：target 低于当前预测，更新使 $V_w(S_t)$ 倾向减小。
- $|\delta_t|$ 越大：在相同学习率下，本次纠偏通常越强。

基础 TD Critic 只对当前状态预测这一侧求梯度，也叫 Semi-gradient TD（半梯度时序差分）。Stop gradient 只在本次 backward 中固定 target；下一轮仍会用更新后的 Critic 重新计算 target。在 PyTorch 中对应 `detach()`，在 TensorFlow 中对应 `stop_gradient()`。

8.4.5 数值例子：一步 Target 为 7.4 时怎样纠正 Critic

假设一次非终止 transition 得到：

```
R_{t+1} = 2
gamma = 0.9
d_t = 0
V_w(S_t) = 5
V_w(S_{t+1}) = 6
```

则：

$$Y_t^{(l)} = 2 + 0.9 \times 6 = 7.4$$

$$\delta_t = 7.4 - 5 = 2.4$$

Critic 当前把 S_t 估成 5，但一步目标是 7.4，所以它会把 $V_w(S_t)$ 往上调整。注意它不会在一次更新后直接把输出硬设成 7.4，而是由学习率和网络梯度决定移动幅度。

如果同一步是环境真正终止，即 $d_t=1$ ：

$$Y_t^{(l)} = R_{t+1} = 2$$

此时不能把任意终止观测 S_{t+1} 的网络输出接进 target，否则会虚构终止后的未来回报。

8.4.6 Bootstrap 如何把终点奖励传播到更早状态

考虑一条确定性短链：

```
s_0 --reward 0--> s_1 --reward 10--> terminal
```

设 $\gamma=0.9$ ，初始时 $V_w(s_0)=V_w(s_1)=0$ 。

1. 先学习终止 transition： s_1 的 target 是 10，所以 $V_w(s_1)$ 被向 10 拉近。
2. 再学习前一条 transition： s_0 的 target 变成 $0 + 0.9V_w(s_1)$ ，并逐渐接近 9。

环境只在最后给了奖励 10，但 TD 更新借助下一状态价值，把这条信息逐步传回 s_1 和 s_0 。如果所有 target 都在网络更新前一次性算好，一步 TD 可能需要多轮采样或更新才能传得更远；后面的 n-step target 会一次纳入更多真实奖励。

8.4.7 TD Critic 的收益与代价：低方差但有估计偏差

训练目标	等待时间	样本方差	Bootstrap 偏差
完整 G_t	到 episode 结束	高	无
一步 $Y_t^{(l)}$	一步	较低	有，受 $V_w(S_{t+1})$ 误差影响

Critic 提供更及时、方差更低的学习信号，但并不保证估计永远准确。实践中要关注 value loss、预测尺度、explained variance（解释方差，即价值预测解释回报波动的程度）以及是否正确处理终止与截断。

8.5 第二条训练线：Actor 用 TD Residual 更新策略

8.5.1 TD Residual 同时是 Critic 误差和 Actor 学习信号

展开上一节的 δ_t :

$$\delta_t = R_{t+1} + \gamma(1-d_t)V_w(S_{t+1}) - V_w(S_t)$$

对 Critic，它是“目标减预测”的回归误差；对 Actor，它回答：

这次实际动作带来的结果，比当前状态下的平均预期好还是差？

- $\delta_t > 0$: 结果比平均预期好，可以鼓励该动作。
- $\delta_t < 0$: 结果比平均预期差，可以抑制该动作。

8.5.2 为什么 TD Residual 可以估计 Advantage

理论 Advantage Function（优势函数）为：

$$A_\pi(s_t, a_t) = Q_\pi(s_t, a_t) - V_\pi(s_t)$$

动作价值可写成：

$$Q_\pi(s_t, a_t) = \mathbb{E}[R_{t+1} + \gamma(1-d_t)V_\pi(S_{t+1}) \mid s_t, a_t]$$

因此：

$$A_\pi(s_t, a_t) = \mathbb{E}[R_{t+1} + \gamma(1-d_t)V_\pi(S_{t+1}) - V_\pi(s_t) \mid s_t, a_t]$$

如果 $V_w \approx V_\pi$ ，再用一次 transition 近似上面的条件期望，就得到一步 Advantage Estimate（优势估计）：

$$\hat{A}_t^{(1)} = \delta_t$$

帽子表示它是有限样本估计量，上标 (1) 表示只使用一步 TD 信息。它不是真实 advantage：环境采样带来随机性，Critic 不准还会带来估计偏差。

8.5.3 Actor 梯度的理论形式：沿轨迹累加后取期望

令 $\tau \sim p_\theta(\tau)$ 表示整条轨迹由当前策略 π_θ 与环境共同产生。策略梯度的轨迹形式为：

$$g(\theta) = \mathbb{E}_{\tau \sim p_\theta(\tau)} [\sum_{t=0}^{T_\tau-1} \gamma^t \nabla_\theta \log \pi_\theta(A_t \mid S_t) A_\pi(S_t, A_t)]$$

这里：

- $p_\theta(\tau)$ 是当前策略和环境共同诱导的 Trajectory Distribution（轨迹分布）。
- T_τ 是轨迹 τ 的有效长度。
- 先对一条轨迹中的时间步求和，再对所有可能轨迹取期望。
- $g(\theta)$ 是 Actor 目标的期望梯度，不是可以直接精确算出的有限数据 loss。

这条公式明确了 Actor 真正要近似的对象：先累加一条轨迹中各时间步的策略梯度贡献，再对策略和环境可能产生的轨迹取平均。实现中还拿不到真实 A_π ，所以下一节同时要把轨迹期望换成 rollout 样本平均，并把 A_π 换成 $\hat{A}$ 。

8.5.4 Actor Loss 的实现形式：对 Rollout Batch 取样本平均

为了把上一节的理论梯度变成可训练的 Actor loss，用当前策略采样 N 段 rollout。第 i 段长度为 T_i ，总样本数为 $M = \sum_i T_i$ ，于是期望梯度的样本估计为：

$$\hat{g}(\theta) = 1/M \sum_{i=1}^N \sum_{t=0}^{T_i-1} \gamma^t \nabla_\theta \log \pi_\theta(A_{i,t} \mid S_{i,t}) \hat{A}_{i,t}$$

i 是 rollout 编号， t 是该 rollout 内的时间步。除以 M 是取平均；不除只会整体缩放梯度，通常可被学习率吸收，但 batch 大小时更新尺度会变化。

自动微分框架通常最小化 loss，所以对已采样且本轮视为固定的 B_{roll} ，构造 Surrogate Loss（代理损失）：

$$L_{actor}(\theta; B_{roll}) = -1/M \sum_{i=1}^N \sum_{t=0}^{T_i-1} \gamma^t \log \pi_{\theta}(A_{i,t} | S_{i,t}) \text{sg}(\hat{A}_{i,t})$$

对这个 loss 求梯度，正好得到 $-\hat{g}(\theta)$ 。部分实现不在 Actor loss 外显式写 γ^t ，而是采用平均状态访问分布的目标约定，或已把折扣影响放进 return/advantage；阅读代码时要检查其目标定义，不能把两种写法机械混用。

这里停止 advantage 的梯度，是为了让 Actor loss 只修改策略参数 θ 。Critic 参数 w 仍由自己的 value loss 更新，两个模块通过数值信号协作，而不是让两条反向传播路径混在一起。

8.5.5 一条 Transition 怎样产生两次参数更新

沿用 $\delta_t=2.4$ 的例子，同一个 transition 产生两次职责不同的更新：

1. **Critic 更新**：把 $V_w(S_t)$ 往一步 target 7.4 拉近，改进以后对状态的平均预期。
2. **Actor 更新**：把 2.4 当作正的动作权重，提高当时所选动作 A_t 的概率或概率密度。

如果 $\delta_t=-2.4$ ，Critic 会下调状态价值，Actor 则会降低该动作的概率。由此可见，Critic 不是只“列一个 loss”：它先学习价值尺度，再把“实际结果相对价值尺度的偏差”交给 Actor。

8.6 从一步 TD 扩展到 n-step：统一改变两条训练线

8.6.1 一步 Target 过度依赖不准确的 Critic

一步 TD 更新快、方差较低，但高度依赖 $V_w(S_{t+1})$ 。Critic 训练早期不准时，一步 target 和 Actor 收到的 advantage 都可能被带偏。

令 n 为 Bootstrap Horizon（自举跨度），即先观察多少步真实 reward，再接上价值预测。 n 是正整数，上标 (n) 表示“向前看 n 步”，不是乘方：

$$Y_t^{(n)} = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{n-1} R_{t+n} + \gamma^n V_w(S_{t+n})$$

如果这 n 步内真正终止，终止后的 reward 和 bootstrap value 都不再加入。

8.6.2 同一个 n-step Target 同时训练 Critic 和 Actor

对 Critic， $Y_t^{(n)}$ 是新的回归标签：

$$L_{critic}^{(n)} = \mathbb{E}_{B_{roll}} [(V_w(S_t) - \text{sg}(Y_t^{(n)}))^2]$$

对 Actor，减去当前状态价值后得到 n-step advantage estimate：

$$\hat{A}_t^{(n)} = Y_t^{(n)} - V_w(S_t)$$

然后把 $\hat{A}_t^{(n)}$ 代入 Actor loss。也就是说， n 不只决定 Actor “看多远”，还决定 Critic 用什么时间尺度的监督目标训练。

8.6.3 n 控制真实奖励与 Bootstrap 的比例

n 的大小	真实 reward 比例	Bootstrap 依赖	样本方差	Bootstrap 偏差
小	少	强	较低	较强
大	多	弱	较高	较弱
到 episode 末尾	完整 G_t	无	最高	无

n-step 不是独立的新算法，而是 Critic target 和 advantage estimate 的共同时间尺度选择。

8.6.4 Rollout Truncation 仍然需要 Bootstrap

并行采样常固定收集 T 步 rollout。到达 rollout 边界只表示 Truncation（截断：这批采样暂时停止），不一定表示环境真正结束。

- 真正 terminal：后续价值为 0。
- 仅 rollout truncation：通常仍用 $V_w(S_{t+T})$ bootstrap。

如果把所有 rollout 边界都当 terminal，会系统性低估末端状态价值，也会同时污染 Critic target 和 Actor advantage。

8.7 GAE：加权组合不同长度的 Advantage Estimate

8.7.1 从连续 TD Residual 构造 GAE

一步 advantage estimate 是：

$$\hat{A}_t^{(l)} = \delta_t$$

如果下一步也持续比预期好， δ_{t+1} 也应该回传到当前动作，只是影响要衰减。这里 λ 是 GAE Lambda（GAE 衰减参数），取值通常在 $[0, 1]$ ：

$$\delta_t + \gamma \lambda \delta_{t+1}$$

继续累加得到 GAE：

$$\hat{A}_t^{GAE(\gamma, \lambda)} = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}$$

这里 l 是从当前时刻向后偏移的步数： $l=0$ 对应 δ_t ， $l=1$ 对应 δ_{t+1} 。上式写成无穷和是理论表达，有限 episode 或 rollout 会在边界处停止。

有限 rollout 中可从后往前递推：

$$\hat{A}_t^{GAE} = \delta_t + \gamma \lambda (1 - d_t) \hat{A}_{t+1}^{GAE}$$

这里的 $\hat{A}_t^{GAE}$ 与上面的 GAE 无穷和是同一个量，递推式只是更高效的计算方式，不是换回了普通 $\hat{A}_t$ 。这条递推只在 $t+1$ 仍位于当前 rollout 时继续。对 rollout 最后一个已保存时间步，把“下一时刻 GAE estimate”初始化为 0；如果此处只是采样截断而非真正终止， δ_t 中仍保留 $V_w(S_{t+1})$ 的 bootstrap。也就是说：价值可以跨截断边界 bootstrap，但 GAE 递推不能读取 buffer 之外不存在的 residual。

8.7.2 GAE 等价于加权多个 n-step Advantage

GAE 也可理解为对不同 n-step advantage estimate 做指数加权：

$$\hat{A}_t^{GAE} = (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} \hat{A}_t^{(n)}$$

因此它不是“另一个突然出现的 advantage 定义”，而是把一步、两步、三步等估计平滑组合起来。

▼ 推导过程：为什么两种 GAE 写法等价（点击展开）

先暂时忽略 terminal mask，考虑一段尚未终止的轨迹。TD residual 为：

$$\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$$

第一步：连续 TD residual 为什么会得到 n-step advantage？

一步时直接有：

$$\hat{A}_t^{(1)} = \delta_t$$

两步时，把下一时刻 residual 折扣后加回来：

$$\begin{aligned}\delta_t + \gamma \delta_{t+1} &= [R_{t+1} + \gamma V(S_{t+1}) - V(S_t)] \\ &\quad + \gamma [R_{t+2} + \gamma V(S_{t+2}) - V(S_{t+1})] \\ &= R_{t+1} + \gamma R_{t+2} + \gamma^2 V(S_{t+2}) - V(S_t)\end{aligned}$$

$+\gamma V(S_{t+1})$ 与 $-\gamma V(S_{t+1})$ 正好抵消，剩下的正是 two-step value target 减去当前 value。因此：

$$\hat{A}_t^{(2)} = \delta_t + \gamma \delta_{t+1}$$

同理，中间 value 会逐项抵消，一般的 n-step advantage 可以写成：

$$\hat{A}_t^{(n)} = \sum_{l=0}^{n-1} \gamma^l \delta_{t+l}$$

这里解释了为什么未来的 δ_{t+l} 与当前动作有关：当前动作不仅影响眼前 reward，也影响之后到达的状态和奖励；把后续 residual 折扣传回，正是在估计当前动作造成的多步后果。

第二步：加权多个 n-step advantage 为什么变成 GAE residual 和？

把上式代入 n-step 加权定义：

$$\begin{aligned}\hat{A}_t^{GAE} &= (1-\lambda) \sum_{n=l}^{\infty} \lambda^{n-l} \hat{A}_t^{(n)} \\ &= (1-\lambda) \sum_{n=l}^{\infty} \lambda^{n-l} \sum_{l=0}^{n-1} \gamma^l \delta_{t+l}\end{aligned}$$

固定某个 δ_{t+l} 看它的总权重。只有长度 $n \geq l+1$ 的 n-step advantage 会包含它，所以其系数为：

$$\begin{aligned}(1-\lambda) \gamma^l \sum_{n=l+1}^{\infty} \lambda^{n-l} &= (1-\lambda) \gamma^l \lambda^l / (1-\lambda) \\ &= (\gamma \lambda)^l\end{aligned}$$

因此：

$$\hat{A}_t^{GAE} = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}$$

第三步：为什么实现中可以从后往前递推？

把无穷和的第一项拆出来：

$$\begin{aligned}\hat{A}_t^{GAE} &= \delta_t + \gamma \lambda \delta_{t+1} + (\gamma \lambda)^2 \delta_{t+2} + \dots \\ &= \delta_t + \gamma \lambda [\delta_{t+1} + \gamma \lambda \delta_{t+2} + \dots] \\ &= \delta_t + \gamma \lambda \hat{A}_{t+1}^{GAE}\end{aligned}$$

真正终止时不能继续读取未来项，因此加入 $1-d_t$ ，得到有限 rollout 中使用的递推：

$$\hat{A}_t^{GAE} = \delta_t + \gamma \lambda (1-d_t) \hat{A}_{t+1}^{GAE}$$

这不是第三种 GAE 定义，只是第一种 residual 求和的动态规划写法，计算量从反复求和降为一次反向扫描。

8.7.3 λ 控制 Advantage 回看多远

- $\lambda=0$: $\hat{A}_t^{GAE}=\delta_t$ ，退化为一部 TD。
- λ 接近 1: 纳入更长时间的 residual，更接近长回报。
- λ 越大: 通常 bias 降低、variance 增大。
- λ 越小: 通常 variance 降低、但更依赖 Critic 的短期 bootstrap。

实践中常见 $\lambda=0.95$ ，但它是需要结合任务 horizon、reward 噪声和 Critic 质量调节的超参数，不是理论固定值。

8.7.4 数值例子：三个 TD Residual 得到 GAE

假设连续三个 TD residual 为：

$$\delta_t=2, \delta_{t+1}=1, \delta_{t+2}=-1$$

令：

$$\gamma=0.9, \lambda=0.95, \gamma\lambda=0.855$$

截断到三项：

$$\begin{aligned}\hat{A}_t^{GAE} &= 2 + 0.855 \times 1 + 0.855^2 \times (-1) \\ &\approx 2.124\end{aligned}$$

当前动作先得到连续正面信号，但第三步出现负面 residual，因此最终 advantage 略高于 2，而不是简单只取第一步的 2。

8.7.5 用 GAE Advantage 还原 Critic 的 Value Target

PPO/A2C 实现中经常先算 GAE advantage，再构造 Critic target。记 $\hat{V}_t^{target}$ 为 Estimated Value Target（价值目标估计）：帽子表示它由有限样本估计得到，target 表示它是 Critic 要拟合的标签，不是当前网络输出。

$$\hat{V}_t^{target} = \hat{A}_t^{GAE} + V_w(S_t)$$

原因是 advantage 定义本来就是：

$$A = Q - V$$

所以“value target = advantage estimate + old value estimate”是在把基线加回去。实现时通常保存 rollout 时的 value prediction，避免更新过程中 target 跟着当前网络反复变化。

从下一节开始， $\hat{A}_t$ 不带上标时表示 Generic Advantage Estimate（通用优势估计）：它可以选用一步 TD、固定 n-step 或本节的 $\hat{A}_t^{GAE}$ 。只有从这里开始才允许省略具体估计方法的上标。

8.8 组装 Actor-Critic 的 Actor、Critic 与 Entropy Loss

下面用 $\mathbb{E}_{B_{roll}}$ 简写对当前 on-policy rollout batch 中全部有效 transition 的样本均值；其展开就是 8.5.4 的 i, t 双重求和，不是只对时间下标取概率期望。

本节采用 A2C 实现中常见的 batch-mean 写法，把折扣影响放在 return/advantage 的构造中，因此不在 Actor loss 外重复写 γ^t 。若采用 8.5.3 的严格 start-state discounted objective（初始状态折扣目标），则使用 8.5.4 中显式带 γ^t 的形式。

8.8.1 Actor Loss

$$L_{actor} = -\mathbb{E}_{B_{roll}} [\log \pi_\theta(A_t | S_t) \text{sg}(\hat{A}_t)]$$

它只负责：根据 advantage 的正负调整已采样动作概率。

8.8.2 Critic Loss

$$L_{critic} = \mathbb{E}_{B_{roll}} [(V_w(S_t) - \text{sg}(\hat{V}_t^{target}))^2]$$

它只负责：让价值预测接近一步、n-step 或 GAE 构造出的 value target。

8.8.3 Entropy Loss 只负责维持探索

Entropy 不是 REINFORCE 或 Actor-Critic theorem 必然推出来的项，而是额外加入的探索正则。

策略熵：

$$H(\pi_{\theta}(\cdot|s)) = -\sum_a \pi_{\theta}(a|s) \log \pi_{\theta}(a|s)$$

$H(\pi_{\theta})$ 是 Policy Entropy（策略熵）；离散动作下，它越大表示动作概率分布越分散。

最小化形式可写成：

$$L_{entropy} = -\beta_H \mathbb{E}_{S_{roll}} [H(\pi_{\theta}(\cdot|S_{\nu}))]$$

$L_{entropy}$ 是 Entropy Regularization Loss（熵正则损失）， β_H 是 Entropy Coefficient（熵系数），控制探索正则则在总 loss 中的强度。

作用：

- 防止策略过早变成近乎确定性分布。
- 保留探索。
- 改善 early training 的状态覆盖。

它与第 10 章 Soft Actor-Critic（SAC，软演员-评论家算法）的区别是：

- A2C/PPO 常把 entropy 当 regularization bonus。
- SAC 从最大熵目标重新定义 soft value 和 Bellman target，entropy 是算法核心目标的一部分。

8.8.4 三个训练目标如何组成总 Loss

共享网络实现常写。这里 L 表示优化器最终最小化的 Total Loss（总损失）， c_V 是 Value-loss Coefficient（价值损失系数）：

$$L = L_{actor} + c_V L_{critic} + L_{entropy}$$

三个项的职责不同，不能因为写在一个式子里就认为它们来自同一推导。

8.8.5 Actor 与 Critic 共享或分离参数的取舍

Actor 和 Critic 可以：

- 共享 backbone（特征提取主干），使用 policy head（策略输出分支）和 value head（价值输出分支）。
- 使用完全分离的两个网络。

共享网络：

- 参数更少，表征可复用。
- Actor/Critic 梯度可能相互干扰。
- c_V 的尺度更敏感。

分离网络：

- 优化边界更清楚。
- 参数和计算更多。

8.9 混合动作空间中的 Actor-Critic

第 7 章把联合动作写成：

$$A_t = (K_t U_t)$$

其中 K_t 是离散类型， U_t 是该类型下的连续参数。联合 log probability：

$$\log \pi_{\theta}(K_t U_t | S_t) = \log \pi_{\theta_d}(K_t | S_t) + \log \pi_{\theta_c}(U_t | S_t, K_t)$$

其中 θ_d 是离散动作分支参数， θ_c 是连续参数分支参数；下标 d/c 分别来自 discrete/continuous。

使用 state-value Critic 时，Actor loss 为：

$$L_{actor} = -\mathbb{E}_{B_{roll}} \int (\log \pi_{\theta_d}(K_t | S_t) + \log \pi_{\theta_c}(U_t | S_t K_t)) \text{sg}(\hat{A}_t)$$

同一个 $\hat{A}_t$ 评价整个联合动作。

State-value Critic 的一个优点是只输入状态：

$$V_w(S_t)$$

它不需要枚举或显式表示复杂的联合动作空间。另一类方法会训练：

$$Q_w(S_t K_t U_t)$$

$Q_w(S_t K_t U_t)$ 是 Action-value Critic Estimate（动作价值 Critic 估计）：它同时输入状态、离散类型和连续参数。这种 Critic 能更直接区分离散类型和连续参数，但建模与优化更复杂。

工程上还要注意：

- 只计算实际选择的离散分支对应的连续 log probability。
- 分别监控 Categorical Entropy（类别分布熵）和 Continuous-policy Entropy（连续策略熵），避免一项支配另一项。
- 对离散 Action Mask（动作掩码，用于屏蔽非法动作）和连续参数边界使用一致的采样与 log-prob 计算。

8.10 A2C 与 A3C：采样组织方式

8.10.1 Actor-Critic 是框架，A2C/A3C 是训练组织方式

Actor-Critic 是框架：Actor 用 Critic 的估值信号更新。

A2C/A3C 是具体的数据采样和参数更新组织方式。它们不会重新定义 advantage，而是选择怎样并行产生 rollout、何时聚合梯度。

8.10.2 A3C

A3C 是 Asynchronous Advantage Actor-Critic（异步优势 Actor-Critic）。这里 worker 指独立运行环境并采集 rollout 的工作进程或线程：

1. 多个 worker 各自持有环境副本。
2. 每个 worker 用本地策略采样短 rollout。
3. 计算 n-step return 或 advantage。
4. 异步把梯度应用到共享全局网络。
5. 再从全局网络同步最新参数。

优点：

- 不同 worker 提高状态覆盖和数据多样性。
- 在不使用 replay buffer 的 on-policy 场景中降低样本相关性。

问题：

- 异步更新存在参数陈旧和 nondeterminism（非确定性：相同设置下的更新顺序与结果可能不同）。
- 工程调试较复杂。

8.10.3 A2C

A2C 是 Advantage Actor-Critic（优势 Actor-Critic），可理解为同步版本：

1. N 个环境使用同一版策略并行采样 T 步。

2. 聚合成 $N \times T$ 的 rollout batch。
3. 统一计算 advantage 和 value target。
4. 对 Actor/Critic 做一次或若干次同步更新。

A2C 通常更适合现代 Graphics Processing Unit (GPU, 图形处理器) 的 batch 计算, 结果也更容易复现。

维度	A2C	A3C
Worker 更新	同步聚合	异步写共享参数
参数版本	一批数据基本一致	Worker 可能稍陈旧
GPU 利用	更自然	相对困难
可复现性	较好	较弱
核心 advantage	TD / n-step / GAE 均可	TD / n-step 为经典做法

8.11 A2C 一轮从 Rollout 到参数更新的完整数据流

1. 用当前 policy π_θ 在多个环境采样 T 步。
2. 暂存到 rollout buffer: state、action、reward、terminal indicator d_t 、log probability、value estimate。
3. 在 rollout 末端:
 - terminal $\rightarrow$ bootstrap value = 0
 - truncation $\rightarrow$ bootstrap value = $V_w(\text{last_state})$
4. 从后往前计算 TD residual。
5. 用 n-step 或 GAE 得到 $A_{\hat{t}}$ (优势估计)。
6. 构造 value target = $A_{\hat{t}}$ + rollout-time value estimate (采样时保存的价值预测)。
7. Actor 最小化 $-\log_{\text{prob}} * \text{stop_gradient}(A_{\hat{t}})$ 。
8. Critic 回归 $\text{stop_gradient}(\text{value target})$ 。
9. 可选加入 entropy bonus。
10. 更新后清空 rollout buffer; 下一批数据用更新后的当前策略重新采样。

这条数据流是理解第 9 章 PPO 的前置输入。PPO 不会重新发明 Critic 或 GAE, 而是重点改变第 7 步的 Actor objective, 并允许对同一 rollout 做有限的多 epoch (多轮遍历) 更新。

8.12 Actor-Critic 的算法边界与行为直觉

8.12.1 REINFORCE with Baseline 与 TD Actor-Critic

维度	REINFORCE with Baseline	Actor-Critic
Baseline/Value 标签	完整 G_t	TD、n-step 或 GAE target
Bootstrap	否	通常是
更新等待	通常到 episode 结束	短 rollout 后即可
方差	较高	较低
估计偏差	无 bootstrap bias	有 Critic bias
学习模块	Policy + 可选 MC baseline	Actor + TD Critic

8.12.2 游戏 Agent: TD Residual 如何改变动作概率

- Actor 输出移动、攻击、防御的动作分布。

- Critic 估计当前局面的 $V_w(s)$ 。
- 如果一次动作得到 $\delta_t > 0$ ，说明结果超出当前局面平均预期，Actor 提高该动作概率。
- GAE 会把后面数步持续发生的正负 TD residual 传回当前动作。

8.12.3 为什么 Critic 不直接决定动作

在 state-value Actor-Critic 中：

- Critic 只输出 $V_w(s)$ ，不负责在动作中取 argmax。
- Actor 输出动作分布并真正执行。
- Critic 的职责是提供相对学习信号，而不是替代 Actor。

这与 Deep Q-Network (DQN，深度 Q 网络) 直接通过 $\arg \max_a Q(s,a)$ 选动作不同。

8.13 本章面试高频问题

8.13.1 Actor 和 Critic 分别做什么？

Actor 输出并更新策略；Critic 估计状态或动作价值，为 Actor 构造较低方差的 advantage estimate。

8.13.2 TD Residual 为什么能估计 Advantage？

因为在真实 V_π 下：

$$\mathbb{E}[\delta_t | s_t, a_t] = Q_\pi(s_t, a_t) - V_\pi(s_t) = A_\pi(s_t, a_t)$$

实际用近似 V_w 时还会带有 Critic 误差。

8.13.3 n-step 与 GAE 的关系？

n-step 选择一个固定 bootstrap horizon；GAE 对多个 n-step advantage estimate 做指数加权，避免只押注一个时间尺度。

8.13.4 为什么 Actor Loss 要对 Advantage Stop Gradient？

Advantage 是提供给 Actor 的样本权重。Actor loss 应只更新策略概率，Critic 应由自己的 value loss 更新；否则两类目标的梯度会意外耦合。

8.13.5 A2C 和 A3C 区别？

A2C 同步聚合多个环境的 rollout 后更新；A3C 让多个 worker 异步更新共享参数。它们共享 Actor-Critic 基础，主要区别是并行与参数同步方式。

8.13.6 Entropy Bonus 是 Actor-Critic 必需的吗？

不是。它是常用探索正则。去掉它仍然是 Actor-Critic，但策略可能更快变得确定、探索不足。

8.14 本章练习

1. 从 $A_\pi = Q_\pi - V_\pi$ 推导为什么一步 TD residual 可估计 advantage。
2. 对给定的 $R_{t+1}=1$ 、 $\gamma=0.9$ 、 $V(s_t)=4$ 、 $V(s_{t+1})=5$ ，计算一步 target 和 TD residual。
3. 写出三步 value target 和对应 advantage estimate。
4. 解释 GAE 中 $\lambda=0$ 与 $\lambda \rightarrow 1$ 的含义。
5. 说明 terminal 与 rollout truncation 在 bootstrap mask 上为什么不同。
6. 写出混合动作 (k,u) 的 Actor loss。
7. 按数据流解释 A2C 一轮 rollout 到更新的全过程。
8. 理论策略梯度为什么要对轨迹取期望并对时间求和？实现中怎样近似？
9. Rollout buffer 与 replay buffer 有什么区别？基础 A2C 为什么不直接使用历史 replay 数据？
10. Critic 的 δ_t 为正时，参数怎样更新？稀疏的终点奖励又怎样传播到更早状态？

▼ 参考答案（点击展开）

1. **TD residual 与 advantage**: $Q_{\pi}(s_t, a_t) = \mathbb{E}[R_{t+1} + \gamma(V_{\pi}(S_{t+1}) | s_t, a_t)]$ 。减去 $V_{\pi}(s_t)$ 后，得到 $\mathbb{E}[\delta_t | s_t, a_t] = A_{\pi}(s_t, a_t)$ 。用一次 transition 和近似价值 V_w 时， δ_t 是带采样噪声和 Critic bias 的估计。
2. **数值题**：未终止时 $Y_t^{(1)} = 1 + 0.9 \times 5 = 5.5$ ， $\delta_t = 5.5 - 4 = 1.5$ 。结果高于当前状态预期，Actor 应提高已采样动作概率。
3. **三步目标**： $Y_t^{(3)} = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 V_w(S_{t+3})$ ；对应 $\hat{A}_t^{(3)} = Y_t^{(3)} - V_w(S_t)$ 。若三步内真正终止，终止后的项和 bootstrap value 为 0。
4. **λ 两端**： $\lambda=0$ 时 GAE 只保留 δ_t ，等于一步 TD advantage； λ 趋近 1 时更长时间的 residual 权重变大，在 episodic 且边界处理正确时更接近 Monte Carlo advantage。前者通常方差低、bias 高，后者相反。
5. **Terminal vs truncation**：terminal 表示环境过程真正结束，环境定义的未来 return 为 0；truncation 只是采样批次或时间上限结束，真实过程可能继续，因此通常需要用最后状态的 V_w bootstrap。
6. **混合动作 loss**：若 $\pi(k, u | s) = \pi_d(k | s) \pi_c(u | s, k)$ ，则 $L_{actor} = -[\log \pi_d(k_t | s_t) + \log \pi_c(u_t | s_t, k_t)] \text{sg}(\hat{A}_t)$ 。只计算实际选择的离散分支对应的连续参数 log probability。
7. **A2C 数据流**：多个环境用同一策略同步采样 T 步，保存动作 log probability 和 value；按 terminal/truncation 决定末端 bootstrap；反向计算 TD residual 和 GAE；构造 value target；Actor 用 $-\log \pi \hat{A}$ 更新，Critic 回归 value target，可选加入 entropy bonus；完成后再用新策略采下一批数据。
8. **理论期望与样本估计**：理论上先把一条轨迹各时间步的 policy-gradient contribution 相加，再对当前策略可能产生的轨迹分布取期望。实现无法枚举轨迹，便用当前策略采样 N 段 rollout，对所有 $M = \sum_i T_i$ 个有效 transition 的贡献求和并除以 M ，得到 Monte Carlo 样本估计。
9. **两类 buffer**：rollout buffer 临时保存当前策略刚采到的连续数据，用来计算 TD residual、GAE 和本轮 loss，更新后通常清空；replay buffer 长期保存多个历史策略的数据并反复随机采样。基础 A2C 的梯度要求数据来自当前策略，直接混入历史 replay 数据会造成采样分布不匹配。
10. **Critic 更新与奖励传播**：对单样本有 $w \leftarrow w + \alpha_w \delta_t \nabla_w V_w(S_t)$ 。 $\delta_t > 0$ 时，更新使当前状态价值倾向增大。终点奖励先修正终点前状态的价值；该价值再进入更早状态的 bootstrap target，使奖励沿 Bellman 递推逐步向前传播。n-step target 可以在一次目标构造中传播更多步。